# Karan Gupta

B.Tech Instrumentation & Control Engineering (2nd year) · NIT Jalandhar
guptakaran0720@gmail.com · github.com/guptakaran20 · guptakaran0720.vercel.app

PROJECT PORTFOLIO
CV & Edge AI Internship
BALRAM Lab, IIT Roorkee

## 01 ILEED — Real-Time Face Recognition on Live Camera Streams

Full Stack Developer & App Lead, XCEED, NIT Jalandhar · June 2026 – present · deployed in production

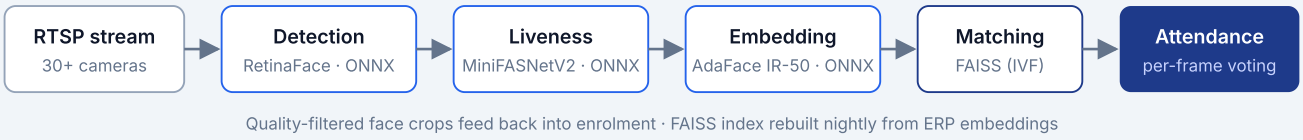
Python · ONNX · RetinaFace · AdaFace IR-50 · MiniFASNetV2 · FAISS · DBSCAN · RTSP · Node.js · Linux

| 1,000+ | 30+ | 3 | 29 |
|---|---|---|---|
| students served in production | live RTSP classroom cameras | ONNX models in the inference path | missed students recovered across 9 periods |

### OVERVIEW

ILEED takes classroom attendance automatically from live camera feeds. A Python ML service detects faces, rejects spoofs (photos or screens), converts each face into an embedding and matches it against enrolled students in a FAISS vector index. A Node.js backend, React web client and mobile app are built on top of it.

### INFERENCE PIPELINE




Quality-filtered face crops feed back into enrolment · FAISS index rebuilt nightly from ERP embeddings

### KEY CONTRIBUTIONS

| | |
|---|---|
| **Model integration** | Integrated the FAISS vector index with the Node.js backend for real-time recognition on RTSP streams, and wired the RetinaFace, AdaFace IR-50 and MiniFASNetV2 ONNX models into the ML service. |
| **Recognition accuracy** | Students were going unmarked when DBSCAN merged different people's face clusters. I added per-frame voting, which recovered **29 students across 9 class periods**. |
| **Inference correctness** | Found and fixed incorrect input preprocessing for the MiniFASNet liveness model. |
| **Systems debugging** | Fixed a FAISS OpenMP deadlock, and a crash when rebuilding the IVF index once it grew past 1,000 vectors. |
| **Data-quality loop** | Built a feedback loop that keeps only clear face crops, scored by Laplacian sharpness, landmark-based head yaw/pitch, detector confidence and the margin over the runner-up match, with automatic embedding generation from the college ERP and nightly FAISS rebuilds. |
| **Deployment & GPU ops** | Shipped one-click deploys with health checks and automatic rollback, and a GPU inventory dashboard with MIG/VRAM telemetry. |

### PRACTICAL SKILLS APPLIED

**Deep-learning inference**
ONNX model integration, input preprocessing, detection → liveness → embedding chains

**Image quality & geometry**
Laplacian sharpness, head pose (yaw/pitch) from facial landmarks

**Vector search & clustering**
FAISS IVF indexes, confidence margins, DBSCAN, temporal voting

**Production systems**
Python, Node.js, Linux, multi-threaded (OpenMP) debugging, GPU telemetry

**Karan Gupta**
guptakaran0720@gmail.com · github.com/guptakaran20



## 02 EventFlow — Fault-Tolerant Distributed Pipeline Engine

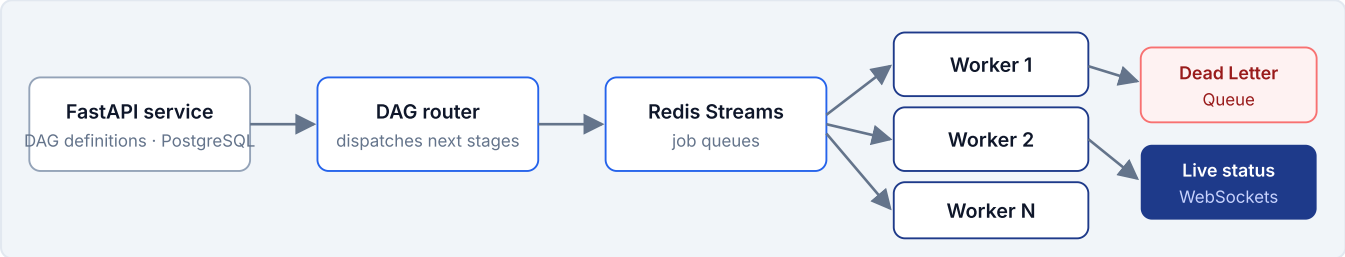
Independent project · github.com/guptakaran20/EventFlow · live demo: goeventflow.vercel.app

Python | FastAPI | Redis Streams | PostgreSQL | gRPC | WebSockets | Docker

### OVERVIEW

EventFlow runs multi-stage workflows, defined as directed acyclic graphs (DAGs), across a pool of distributed workers. It was designed around one question: what happens when a stage or a worker fails mid-run? The same concern applies to any real-time perception pipeline, where a single slow or crashed stage must not stall the whole system.

### ARCHITECTURE




Workers send heartbeats; jobs left pending by a crashed worker are reclaimed (XPENDING/XCLAIM) and retried, and stages that keep failing go to the Dead Letter Queue.

### KEY CONTRIBUTIONS

| | |
|---|---|
| **Pipeline design** | Architected DAG routing over distributed Redis Streams workers, with automatic retries and a Dead Letter Queue for stages that keep failing. |
| **Fault tolerance** | Added worker heartbeats and stuck-job recovery using Redis XPENDING/XCLAIM, so work owned by a crashed worker is picked up by another instead of being lost. |
| **Observability** | Streamed live job status to the UI over WebSockets; internal services communicate over gRPC. |
| **Deployment** | Containerised the whole system with Docker, backed by PostgreSQL. |

### PRACTICAL SKILLS APPLIED

**Pipeline engineering**
DAG scheduling, message queues, retries and dead-letter handling

**Reliability**
Heartbeats, failure detection, recovery of in-flight work

**Backend**
Python, FastAPI, PostgreSQL, gRPC, WebSockets

**Infrastructure**
Docker, Linux

**Relevance to the BALRAM lab project.** ILEED gave me hands-on experience running ONNX vision models on live camera streams in production, and EventFlow is how I approach building multi-stage pipelines that keep running when a stage fails. I would like to build on this with stereo vision, camera calibration, 3D perception and TensorRT optimisation on NVIDIA Jetson Orin. As an Instrumentation & Control student, sensors, calibration and real-time systems are close to my coursework.